

ソフトウェアデザイン 2026

教師用補足・公開解説

第2～15回

情報システム学科 3年生 選択2単位

授業で案内する提出先・期限に従って使用する。

演習のフォルダ構成は本教材の推奨例。

第02回 教員用補足：Pythonを自分で確かめる

学生用本編は lectures/02.md、自習演習は exercises/02.md。自習演習は、各例の予測・実行・理由の確認を単独で進められる構成です。確認問題の理由つき解答を学生版にも載せています。この資料は、提出運用と理解の観察、支援の順序を扱います。正式な授業時間や点数配分は設定しません。

学習の進め方

まず、Python 3.12以上を使っていることと、自分のファイルを保存して実行できることを確認します。コード・出力・ターミナルのコマンドの区別ができていない段階では、構文の説明を増やす前に、短いprintの例を保存・実行する流れへ戻します。

学生には、実行前の予測を1行でも残すよう促します。予測の正誤だけで評価せず、実際の結果との違いを説明し直せるかを観察します。解答を先に読んだ学生には、値や条件を1つだけ変えた入力で、再度予測してもらいます。

全ての項目を同じ速度で進める必要はありません。値・代入・分岐・反復・関数の基礎を確認してから、共有とコピー、入力検証へ進めます。自習で止まった場所は、節番号・入力・予測・結果を示せると、支援しやすくなります。

概念ごとの観察と支援

概念	観察する説明	誤解があったときの支援
値と型	整数200と文字列"200"を区別する	同じ表示でも加算・連結が違う例を実行
代入	右辺を評価して名前を結び直す	priceとoriginalが何を指すか書く
演算・比較	優先順位、= と == を区別する	括弧を付けた式と条件式を比較
if	字下げが実行範囲を決める	quantityを0と2に変えて分岐を追う
for	totalが処理済み要素の合計になる	反復ごとの加算前後を表にする
while	条件を再評価し、終了へ近づく	countの更新と条件の対応を示す
list	0始まり、要素数と末尾位置を区別する	3要素の位置を0、1、2で書く
共有・コピー	結び直しとオブジェクト変更を区別する	aliasとcopyの例を別々に実行
dict	キー存在と既定値の意味を説明する	不足quantityを0で受け入れる危険を考える
関数	引数、戻り値、終了を区別する	print版とreturn版の呼び出し元を比較
スコープ	ローカル名の変更と共有入力の変更を分ける	整数引数とリスト引数を比較
エラー	種類・行・その時点の値を確認する	1つのエラー例だけを実行して追う

この表は理解の観察用であり、正式な採点表ではありません。

確認問題の補足

学生用の解答に加えて、次を押さえます。

- 代入は、後の更新に追従する式を保存する操作ではありません。整数を再代入する例と、同じリストの要素を変える例を混ぜないで説明します。
- 型変換は元の値を必ず書き換える処理ではありません。count = int(text) ではtextはstrのままです。input() の戻り値もstrです。

- / の整数同士の結果もfloat。// は床除算で、負数に対して常に0方向へ切り捨てるわけではありません。基本例は正の整数とし、負数は補足にします。
- and ・ or は短絡評価します。一般にはオペランドを返すので、常にTrue/Falseだけを返すと説明しないようにします。学生例は比較式を組み合わせているのでboolになります。
- range(3) は0、1、2の3回。forのループ対象はリストではありませんが、この回は要素を順に取り出す例から確認します。
- 空入力の合計0は、初期値と0回の反復から説明できます。単に「空は特別扱い」と暗記させる必要はありません。
- copy() は外側のリストを作り、参照先まで複製しません。入れ子の辞書を変更する例で、何が共有されているかを確認します。
- returnなしの関数もNoneを返します。「戻り値が全く存在しない」とは言いません。returnを実行したら、その呼び出しの後続行へ進みません。
- ローカルな名前への再代入は外側の同名変数を変更しません。ただし、関数へ渡したリストや辞書自体の変更は、呼び出し元へ見えます。
- 型注釈だけでは入力を拒否しません。利用者の入力検証をassertだけに任せず、ifとraiseを使います。

エラーの学習支援

最初は、エラーを消すことより、何を確認する必要があるかを言えることを重視します。NameErrorなら名前の定義・綴り、TypeErrorなら演算する型、ValueErrorなら許可する値、IndexErrorなら位置、KeyErrorなら必須キーに戻ります。

構文エラーはコードの解釈段階で止まり、実行中の例外は到達した処理で発生します。字下げを変えると実行範囲が変わる点も、単なる見た目として扱わないようにします。エラー文の正確な全文や矢印表示はPythonの版で差があるため、種類と原因の説明を中心に確認します。

意図したValueErrorで停止する例は、正常終了の例と分けて示します。CLIのエラー表示を、そのまま本人の失敗とみなさないようにします。無限whileを試してしまった場合は、Control+Cで止め、終了条件と更新へ戻します。

補助デモの使い方

`python3 examples/02-basics/concepts.py` CASE で概念ごとの例を実行できます。ケース名は学生資料の表に示しています。CASE未指定・未知のCASEはCLIの使い方のエラーとして終了コード2。各デモの正常終了は0です。exceptionsは意図した例外を内部で捕捉するため、例外を表示してもCLIとしては0で終了します。

tracebackの表示は実際の例外から得た抜粋であり、絶対パス・行番号を省略しています。学生が自分のコードのエラーを読む際は、通常のtracebackのファイル名・行番号へ戻ります。デモの抜粋を完全なエラー全文だと扱わないようにします。

学生本文の短例とデモでは、入力や表示の位置が異なります。例えば本文のwhileは更新前のcountを表示し、while-updateは更新前後を1行に表示します。結果が違って見える場合は、同じ仕組みのどの時点を表示したかを比べます。typesの0.1+0.2などは発展の観察とし、注文の円単位の整数仕様と分けます。

注文集計の契約

学生課題の `summary(orders) -> int` は、注文辞書のリストを受け取り、円単位の総額を整数で返します。公開例の `summarize` は品目別数量を含む辞書を返す別の関数なので、課題の戻り値と混同しないよう説明します。

必須条件は次のとおりです。

- 入力全体はlist、各要素はdict。
- name、price、quantityを持ち、nameはstr。空文字も受け入れる現行契約。
- priceとquantityは0以上の厳密なint。bool、float、strを拒否。
- 空入力の総額は0。数量0・単価0は有効。
- キー不足、型違い、負数は理由が分かるValueError。
- 入力のlist・dictを変更しない。重複商品名の各行も加算する。

boolはintの派生なので、`isinstance(value, int)` だけでは契約を満たしません。厳密型の判定は `type(value) is int` などで行います。型を確認してから負数を比較すれば、文字列の比較によるTypeErrorを避けられます。検証方法は複数の書き方を認めますが、契約の例外型を揃えます。

解説例と検証の目安

既定の公開デモはノート200円×2とペン120円×3、総額760円。`summary([])`は0。数量0ではその行の小計0。商品名が同じでも、各行の価格で小計を計算します。

正常・空・数量0・負数・数の文字列・小数・bool・キー不足・入力全体の型違い・要素の型違いを確認します。テストで期待値を同じ実装ロジックから計算し直すと、同じ誤りを見逃しやすいので、小さい値を手で求めます。

入力非変更は、入れ子のdictを含むコピーと比較します。`before = orders`は同じオブジェクトなので不適切。`orders.copy()`も内部dictの変更を見逃します。検証には`deepcopy`を使えますが、集計関数本体に入力のコピーを必須とするわけではありません。入力を読むだけで合計を作れば非変更を保てます。

戻り値の型を観察するときは、760.0やTrueを整数と等値比較するだけで通していないかにも注意します。今回の戻り値はintです。

口頭確認の問い

本人のコードで具体的な入力を指し、次を確認します。

1. 2件目を処理する前後で、totalは何から何へ変わりますか。
2. returnをforの内側に置くと、何件まで処理しますか。
3. 空入力ではどの行を実行しますか。
4. quantityが0、True、"2"なら、それぞれどこを通りますか。
5. 入力のdictを変えていない根拠はどこですか。
6. AIの提案を採用した箇所を、どの入力で確認しましたか。

「わかりましたか」という問いだけで理解を判断しません。説明に詰まった場合は、入力を1件へ減らす、状態を表にする、表示する値の型を確認するといった支援を行います。

提出運用

正式な提出先・期限・点数配分・参照条件は、実授業で案内します。過年度の規則を現年度の運用と推定しません。学生本文の提出案内は末尾に短くまとめ、授業の中心を学習内容に置きます。

推奨成果物は `submission/02/summary.py`、`test_summary.py`、短いREADME（実行方法）と`reflection`（予測・修正・AI支援）。標準`unittest`で正常・境界・異常入力と入力非変更を確認できます。`if __name__ == "__main__":`を用いる場合、その仕組みはコードのimport時と直実行時の違いを短く補足し、計算関数の理解より先行させません。

提出前には実行方法と必要ファイルを確認し、提出後は届いたことを本人が確認できる案内を用意します。未完成の場合も、動く範囲と再現入力を記録するよう促します。正式な再提出の扱いは別途案内します。

AI利用の有無だけで理解を判定しません。使用の用途、提案の採用・修正範囲、本人が予測・実行・説明した内容を観察します。実際の学生の名前・コード・点数・個別講評はこの公開資料へ記載しません。

第3回 教師用補足

AIの利用を推奨し、答えの取得だけでなく学習の変化を観察する。特定サービスの契約・UIを必須にしない。環境が用意できない学生には、自作の提案例と相互質問で検証活動を継続する。

確認問題の解説

1. 出力は仕様を取り違えていても自然に読める。実行例と公式API仕様で確認する。
2. 閉区間で候補1個が残る`left==right`を検査しないため、`while left<right`はこの実装では誤り。半开区間では別の境界定義が必要。
3. 仕様から手計算する、単純な線形探索と照合する、境界ごとに期待値を独立に作る。生成したテストの通過だけで全範囲の正しさは証明しない。
4. 提案の要点、却下の理由、反例・前提・仕様の差を残す。

授業中の観察

学生に未使用の1要素・2要素入力を渡し、探索の`left/right`を説明してもらう。修正版を見て理解した学生も、その後に別の反例や変種を説明できれば学習を確認できる。正式評価点はシラバスにないため、本補足では点を割り振らない。

過去教材との対比

2025のPython基礎は複数回にまたがり、AI学習回は独立していなかった。2026正本に従い、第2回のコードを材料としてAI学習・検証・振り返りを独立した第3回に配置した。過去の成績割合や提出条件は継承していない。

公開教材として自作した例のみを扱い、実際の学生回答・AIチャット・採点情報を載せない。

第04回 教員用：データ構造とアルゴリズム

学生配布用スライドは lectures/04.md、演習は exercises/04.md。ここは解答と学習支援用。正式な授業時間・課題の点数配分を設定する資料ではない。進行は受講者の状態に合わせて、概念説明、実演、段階演習、確認を組み合わせる。

この回の位置付け

2026年の計画に従い、メモリサイズ、計算量、キュー、スタック、ソート、線形探索、全探索、二分探索、DFS、BFS、木を1回で概観する。すべてを同じ深さで実装させるのではなく、処理順の予測、二分探索の前提、操作数の増え方を到達の中心にする。

前回のAI支援による開発へ接続し、「提案されたコードが正しい条件」と「入力が増えたときの費用」を自分で確認させる。

実演時の注意

Python 3.12以上を事前確認する。メモリのbyte数は環境差があるため、講師の出力への一致を正解条件にしない。sys.getsizeof は直接のオブジェクトの大きさを返し、参照先の全体を再帰的に数えない。浅いコピーは要素への参照を複製するため、要素の実体をすべて新しくする処理ではない。

CPythonのlistは可変長配列として扱う。連結リストと説明しない。dequeの両端操作とリスト先頭削除の違いは、要素をずらす必要性で説明する。appendは、リストの容量拡張があるため、ならしの $O(1)$ と瞬間の処理費用を区別する。

スライドの二分探索は半開区間、公開例の二分探索は閉区間を使う。どちらも正しいが、境界の更新方法を混ぜない。不在の戻り値は両方Noneで揃える。

時間は多数の要因で変動する。計算量を推測する主教材は、小さい入力の操作数モデルにする。memory_complexity.py の回数は式から計算したモデル値であり、実行した比較を逐一計測した値ではない。巨大な二重ループの結果をメモリに蓄積する演習は行わない。

確認問題の解説

1. 二分探索は、中央との大小関係から半分を除外する。未整列では、目的の値が除外側に存在し得る。例 [8, 2, 6, 4] で8を探すと、昇順の仮定が成立せず見落とせる。

2. n 件から2つの位置を $i < j$ で選ぶ候補数は $n(n-1)/2$ 、増え方は $O(n^2)$ 。同じ位置を許す二重ループなら n^2 。全探索一般が常に $O(n)$ または $O(n^2)$ とは限らず、候補集合次第。
3. リスト末尾の `pop` を使えば LIFO となり、DFS に近い処理順になる。BFS の距離順処理を保証する FIFO が崩れる。
4. `list` 本体には管理情報と要素への参照がある。 `getsizeof(list)` だけでは、別オブジェクトである各要素の全量を説明できない。

演習の答えと支援

- スタック C B A、キュー A B C。
- 自作木の DFS は A B D E C F、BFS は A B C D E F。
- 線形探索 `[8,2,6,4]` の 6 はインデックス 2。整列済みコピー `[2,4,6,8]` の 6 も 2 だが、同じ数になったのはこの入力による。別の値で元の位置と整列後の位置が異なることを示す。
- 二分探索の区間 `[lo,hi)` は、答えがあればその区間内にあるという条件を保ちつつ縮小する。空列は `while` に入らず `None`。
- $n(n-1)/2$ の候補数は、 $n=3,4,5$ で 3,6,10。

初学者には、比較した位置と `lo/hi` を紙で追わせる。理解が進んだ学生には、重複の先頭を返す版や、ソートを含む複数検索の総コストを検討させる。

よくある誤解

「DFS/BFS には整列が必要」は誤り。列の二分探索と区別する。隣接順や子の順は訪問順を変えるが、整列が探索の成立条件ではない。通常的全走査は $O(V+E)$ 、木では $O(V)$ 。再帰 DFS の呼出し領域は高さに依存する。BFS のキューは幅に依存する。

「 O 記法が秒数そのもの」「動的メモリ管理はすべて GC と同義」「組込みソートの 1 回の実測で $O(n^2)$ が証明できる」といった説明を避ける。

口頭確認と到達の目安

「未整列で二分探索を使える理由はありますか」「区間が 1 要素でも終了しますか」「BFS のキューにはどの深さの頂点が残っていますか」と、入力に戻れる問いを使う。

到達の目安は、構造の処理順を予測できること、二分探索の前提と終了条件を説明できること、操作数とメモリの測定対象を区別できること。誤りがあっても、再現入力から説明を修正できるかを確認する。正式な評価配点は担当教員が別途案内する。

第05回 教員用：グラフの設計

学生配布用は lectures/05.md と exercises/05.md。解答や説明例を含む教員用資料。正式な時間・採点配分は設定しない。

到達目標と進行

頂点・辺・向き・重みを要求から決め、BFSの発見時記録と親の復元へつなぐ。グラフ理論の網羅より、次回の経路課題の仕様を説明できることを重視する。実例の駅名や路線図は転載せず、架空校内マップでモデルの範囲を話す。

説明、図と隣接リストの照合、手作業のキュー追跡、入力変更、確認問題の順で進める。正式な所要分数を固定しない。

実演の正解

examples/05-graphs/graphs.py のグラフはA-B、A-C、B-D、C-D、孤立X。AからDのBFS経路はA B D。Aから到達する集合はA B C D、連結成分は $\{A, B, C, D\}$ と $\{X\}$ の2つ。

Aの隣接順をB、CとしたBFSの追跡例：開始キューA、Aの取り出し後B C、Bの取り出し後C D、Cの取り出し後D。Dの親はBのまま。探索をgoal発見時に止める実装では、最後まででの訪問順を求める実演と出力が異なるので、停止条件を確認する。

次数の説明

今回の単純無向グラフでは次数は隣接頂点数、Aは2、Xは0。無向の自己ループを扱う拡張では次数の数え方が変わるので、今回の前提を示す。有向グラフは入次数・出次数と呼ぶ。値の符号を意味しない。相互に有向辺が存在することと、モデル全体を無向と定義することも区別する。

確認問題の解説

1. 一方通行を無向として登録すると、許可されない逆向き移動を返す。元の意味を変える誤モデル。
2. 頂点をキューに追加した時点で発見済みにする。取り出した時点まで遅らせると、複数の親から同じ頂点を重複追加し得る。

3. 重みなし、または全辺の移動費用が同じ正の定数の場合、最少辺数と最少総費用が一致する。時間や料金が辺ごとに異なれば、BFSだけでは最少総費用を保証しない。
4. 関数は既知の孤立頂点への到達不能を `None` で返せる。未知の頂点名は別に検証し、入力エラーとして扱う。

演習の解説

無向辺B-Dを両方向から削れば、A Dの最短経路はA C D。さらにC-Dを両方向から削れば到達不能。片側だけ削ると「無向」の前提を壊す。有向A B、B DではAからDは到達可能、DからAは到達不能。出次数は隣接リスト長、入次数は全ての隣接リストに対象が現れる回数で求める。

同じ長さの最短経路が複数ある場合、1本返す仕様なら、経路文字列の完全一致だけを正解条件にしない。開始・終了、各辺の存在、最少辺数を確認する。教材の固定データは順序を固定してA B Dを期待する。

よくある誤解と支援

訪問済みは循環対策だけでなく、重複した発見を避ける。parentを後から上書きすると、最初の距離の根拠を失う。DFSで経路が見つかることと、その経路が最短であることを区別する。全経路の列挙はこの回のBFS1本返却とは別の仕様であり、候補数が非常に増え得る。

無向グラフの循環検出は直前の親への辺を除外する。単純なvisited判定だけで親への戻りを循環と誤判定しない。有向グラフで到達可能な集合を1つ求めても、強連結の証明にはならない。

理解に詰まった学生には、コードをいったん閉じ、図の辺を両方向の隣接リストへ転記させる。次にキューの先頭を1個ずつ消し、追加した親だけを書く。

口頭確認と到達の目安

「この辺の意味は何ですか」「Dの親がCへ変わらない理由は何ですか」「時間の最短へ仕様を変えると、何のデータが必要ですか」と確認する。

到達の目安は、モデルの前提を言えること、対称な隣接リストを作れること、発見時記録と復元を説明できること。正式な評価は別途案内する。

第06回 教員用：実装と授業内提出

学生版は `exercises/06.md`、解説用の参考実装は `examples/06-route-project/route_reference.py`。学生用スターターは `route_starter.py`。公開参考実装と学生課題を区別し、参照可否・提出方法・締切は実際の授業で案内する。過年度の提出先、点数配分、協働に関する規則をこの資料から推定しない。

授業の目的と進行

データ構造とアルゴリズムの知識を実装で確認し、授業内に提出する。実装だけで終了せず、正常・境界入力、成果物整理、提出完了の確認まで扱う。正式な授業時間は未設定。進行の状況を見て、状態追跡の支援と実装支援を分ける。

課題は無向・重みなしグラフで最少辺数の経路1本を返す。全経路の列挙や重み付き最短経路は必須要件にしない。

APIとCLIの契約

`route_data.py` は `GRAPH` と `STATIONS` を提供する。`find_route(graph, start, goal)` は経路のlist、到達不能時はNoneを返す。未知の駅名や無向辺の不整合は`ValueError`。入力検証をBFSの前に実行する。CLIはSTART GOAL、省略時A D、失敗は終了コード2。

学生版は同駅と隣駅のみ回答するため、初期のA D失敗は意図した未実装。環境エラーと混同しない。作業先へ学生版とデータを同じディレクトリにコピーする。データのimport失敗なら、先にコピー先と実行パスを確認する。Python 3.12以上で検証する。

参考実装の説明

開始点をdequeに入れ、parentにstart:Noneを置く。FIFOで取り出し、未発見の隣を発見した時点でparentを設定して追加する。goalが発見済みなら、goalから親のNoneまでたどって経路を反転する。到達不能なら復元前にNoneを返す。

親は単に直前の頂点の候補ではなく、最初の発見に使った頂点。1回だけ登録することで重複を避け、最少辺数の層の根拠を保つ。時間は隣接リストと平均的な辞書操作のモデルで $O(V+E)$ 、追加領域 $O(V)$ 。入力の検証もグラフの頂点・辺を調べる。

必須ケースの正解

- A DはA B D、移動数2。
- D AはD B A、移動数2。
- A AはA、移動数0。X XもX、移動数0。
- A BはA B、移動数1。
- A Xは関数でNone、CLIで到達不能と終了コード2。
- UNKNOWN Dは関数でValueError、CLIで入力エラーと終了コード2。

経路は頂点列なので辺数は $\text{len}(\text{path})-1$ 。経路が複数ある自作グラフは、各辺と最少辺数を確認し、1つの文字列に限定しない。

確認問題の解説

1. 開始点の親をNoneにすると、開始点までたどったときの停止条件になる。同時に、開始点を発見済みにする。
2. popはLIFOで奥の枝を先にたどる。FIFOによる距離の層の順序を保てず、最少辺数を保証できなくなる。
3. 同じ出発点と到着点ではpathの長さ1、辺数0。孤立駅でも同駅の移動は0。
4. 未到達のgoalはparentにないため、復元を始めればKeyErrorなどになる。到達不能の仕様に合う値を先に返す。

支援と理解の確認

無限ループなら、未発見判定と登録の時点を確認する。逆向きの経路なら復元後の反転、長い経路ならFIFOとparentの上書きを見る。実装を丸ごと渡す前に、A Dでキューとparentの変化を学生本人に記録させる。

AI利用の有無から理解を断定しない。「提案をどこで確かめたか」「この入力でどの行が実行されるか」と確認する。支援が多かった学生には、同駅・孤立駅の説明など小さい範囲から確認する。

提出確認と到達の目安

推奨成果物は実装・必要データ・検証コード・README・test-results・reflection。提出先に成果物が届いたことを学生本人に確認させる。未完成の場合も、実行できる範囲、再現

入力、残る課題の記録を促す。

到達の目安は、仕様どおりの戻り値、BFSと復元の説明、境界入力の検証、AIや参考実装の参照範囲の記録。これは学習支援の基準で、正式な配点ではない。個別の実採点情報はこの公開資料へ記載しない。

第07回 教員用：講評と質疑

学生版は `lectures/07.md` と `exercises/07.md`。共通講評は全て自作の架空提出例。実学生のコード、氏名、学籍番号、点数は含めない。実際の個別確認は、提出者と担当教員の間で提出内容の理解を扱う。

進行と目的

共通例の仕様違反を反例で説明し、その後に本人の提出物の質疑を行う。共通講評だけで終了させず、各人の設計・検証・AI支援範囲の説明へつなぐ。時間配分は授業状況に合わせ、正式な分数をこの教材では設定しない。再提出の有無や方法は実授業で案内する。

架空例の正解

この回のグラフは第06回と別物。辺はA-B、B-E、E-F、F-D、A-C、C-D、Xは孤立。
`flawed_route.py` の訪問済み付きDFSは、最初にA B E F Dを返し、移動数4を最短と表示する。経路そのものは妥当だが、最少性は誤り。`improved_route.py` のBFSはA C D、移動数2を返す。

A-Cを両方向から削除すれば、4辺のA B E F Dが最短になる。しかし特定の入力で最短が返ることと、全入力の仕様を満たすことは別。`visited`は終了性・重複回避に寄与するが、DFSの最短保証を生まない。

確認問題の解説

1. `visited`があってもDFSは最短を保証しない。隣接順B Cなら、B側の深い枝を先にたどってDを見つける。
2. 妥当な経路は、始点終点と各辺の存在を満たす。最短経路には、もっと少ない辺の経路がないという追加の条件がある。
3. DFSが先に深い経路を見つける一方、別の分岐に短い経路がある小さい入力を使う。同駅や隣駅だけでは違いを検出しにくい。
4. 修正前の反例を再実行し、経路の妥当性と最少辺数を確認する。到達不能、同駅、未知入力も再確認し、新しい不具合が生じていないかを見る。

共通講評の例

「A Dの出力は実在する経路ですが、4辺です。このグラフにはA C Dの2辺があるため、最少辺数の条件を満たしていません。FIFOと最初の親記録を使う版を検討し、同じ入力とA Xで再確認してください。」

これは教材用の例文。実学生の出来や能力を評価した発言ではない。改善の対象・証拠・次の確認を含める。

個別の質疑の問い

- この提出物の最短の定義は何ですか。
- A Dの探索で、2回目の取り出し後のキューと親辞書を示せますか。
- parentへの登録を取り出し後まで遅らせると、何が変わりますか。
- 孤立駅と未知駅で、どの処理と戻り値が違いますか。
- 自作テストは、どの誤りを見つける狙いですか。
- AIまたは参考版を使った箇所を、自分の入力でどう確認しましたか。

本人の第06回グラフと、共通講評の第07回グラフを混ぜない。第06回でA DはA B D、第07回でA DはA C D。

説明に詰まった場合の支援

すぐに解答を代読せず、同駅、隣駅、2段経路へ入力を小さくする。キューから消す要素、追加する要素、parentへ初めて記す組を紙に書く。本人が結果を予測してから実行し、説明を修正する機会を作る。

AIへの質問の詳細な履歴を公開資料へ転載する必要はない。支援の用途、採用範囲、本人による検証の要約を確認する。AIを未使用でも、そのことだけを理解の証拠としない。

到達の目安

仕様と挙動の対応を説明できること、小さい反例で改善点を示せること、本人の提出物の主要状態を追えること、修正の証拠を残せることを確認する。正式な採点配分は別途案内する。個別の理解度・点数・講評記録は公開教材とは分けて管理する。

次回への接続

第08回は電卓を題材に字句解析・構文解析・意味解析。文字列、トークン列、木と表現が変わっても、各段階の入力・出力・不変条件・検証という考え方は共通。木の構造を読めることが、構文のまとまりの理解へつながる。

第8回 教師用補足

2025の言語導入・字句・構文・意味解析の複数回を、2026正本の電卓という題材で統合した。歴史やCPUメーカーの説明に時間を使いすぎず、文字列→トークン→AST→評価を手元で追跡する。授業時間の分数は確定していない。

確認問題の解説

1. 2,+,3はそれぞれ正当なトークン。+の後にfactorが必要だが来るため構文エラー。
2. ASTは計算に必要な構造を表す。空白と括弧の表面情報は、位置情報や具体構文木を別に残す設計もあるが、今回の評価には不要。
3. Binary('-', Binary('-',10,3),2)で5。右結合なら9になり仕様が変わる。
4. 字句/構文の規則だけでは「演算対象が0」を一般に表せない。1/(2-2)もある。静的な定数量み込みで先に発見する設計は可能だが、文法と評価条件を混同しない。

誤解への対応

意味解析と実行は全ての処理系で同じ段階になる訳ではない。教材の電卓は数値のみなので、評価時にゼロ除算を検査する。名前と型の静的検査は後の言語へつなぐ補足。floatの限界は数値表現の設計問題として区別し、計算結果が必ず10進の厳密値になるとは説明しない。

旧数値文法の文字を消費しない再帰は継承せず、digitsと繰り返しで定義した。負数は数値トークンと一体化せず単項演算として扱うため、2-3との区別を説明できる。参考コードは大きな入力向け最適化や深さ制限を主題とせず、小入力の理解を優先する。

理解の確認

木を指して「この子を先に計算する理由」「括弧不足を発見する関数」「テストが通っていても数値の精度に限界がある理由」を聞く。正式な点配分は設定しない。自力実装とstarter拡張のどちらでも、段階・仕様・反例の説明を観察する。

公開補足であり、実際の学生提出物や採点データを含まない。

第9回 教員用ノート

授業の位置付け

LispのREPLと四則演算。2026年度の授業計画に沿う新作教材。PythonホストとLisp方言は教材の設計選択。以下の到達基準は教材案であり、正式な採点配分、実際の成績、提出先や締切を規定しない。

進行の重点

第8回の中置電卓と、第9回の前置S式を比べる。読取と評価を分け、括弧不足とゼロ除算を異なる段階の失敗として扱う。REPLは1行1式であり、複数行入力機能は今回の条件に含めない。

解答と説明

- $(+ 1 (* 2 3))$ は $(* 2 3)$ が 6、外側が 7。
- $(* (+ 2 3) (- 9 4))$ はそれぞれ 5、最後に 25。
- $(/ 8 2)$ は数値として 4。整数・浮動小数点の表示差は契約として確認する。
- 括弧不足と余分な閉じ括弧は読取の失敗。未知の演算子とゼロ除算は評価の失敗。
- REPLで $(/ 1 0)$ の後に $(* 4 5)$ が 20 になることが回復の証拠。

読取器は入れ子のリストを返し、評価器はそのリストの引数を再帰的に評価する。授業の木は自作図であり、原資料の図の転載ではない。

よくある誤解

「括弧を捨ててから空白分割すればよい」では入れ子の構造が失われる。「エラーはすべて構文エラー」では対処箇所を区別できない。「先頭は常に通常の関数」では次回の特種形式を扱えない。今回は四則に限定し、一般化し過ぎない。

口頭確認

$(* 2 (+ 3 4))$ を渡し、木を描いて結果 14 の理由を説明させる。閉じ括弧を1つ減らし、どの段階で何を検出するか追問する。例外をループの外で捕まえた場合、次の入力を受けられるか確認する。

到達基準案

正常な入れ子の式を計算でき、読取・評価・表示をコードで示せる。失敗入力の後にも計算でき、EOF終了を説明できる。値だけを答えた場合は、引数を評価する順序を追問する。

支援の順序

数値1個、2引数加算、入れ子、エラー回復の順に縮める。読取器の内部値を先に観察し、構文と演算の不具合を同時に直させない。AI利用の確認では提案を採用した反例と本人の説明を聞く。

参照と検証

完成参照は `examples/09-lisp-arithmetic/lisp.py`。`starter.py` は既存square組込みへ負数の入力検証を追加するウォームアップ用で、本課題全体の穴埋めではない。

`exercises/09.md` の受入例と、本人の作業用コードの結果を比較する。完成参照を使った範囲を明記させる。

第10回 教員用ノート

授業の位置付け

変数と関数の定義。2026年度の授業計画に沿う新作教材。PythonホストとLisp方言は教材の設計選択。以下の到達基準は教材案であり、正式な採点配分、実際の成績、提出先や締切を規定しない。

進行の重点

`define` と `lambda` を分離し、通常に関数適用と特殊形式を区別する。定義時環境と呼出し時環境を、実例で繰り返し確認する。自分の版の環境構造を描いてから参照と比較させる。

解答と説明

- `(begin (define x (+ 2 3)) x)` は 5。名前の構文 `x` 自体を評価せず登録する。
- `square 6` は 36。本文の評価は呼出し時まで待つ。
- `make-adder 5` が返した関数は `x = 5` の環境を保持し、`add5 3` は 8。
- 外側の `x = 100` と仮引数 `x = 4` は別の環境。`f 4` は 5、外側の `x` は 100。
- `(if #f (/ 1 0) 7)` は 7。非選択枝は評価しない。
- `(quote (+ 1 2))` はリストを返す。内部の `+` の名前検索も計算も行わない。
- `fact 0` は 1、`fact 4` は 24。現在の引数環境の外側で `fact` 自身を探す。

本教材の偽値は `#f`。Pythonの真偽判定をそのまま流用すると、`0` や空リストを偽にしてみよう。

よくある誤解

「関数は本文だけ保存する」では自由変数を解決できない。「呼出し元の環境を外側にする」はレキシカルスコープと異なる。「`if`は引数を全評価する組込み」で非選択枝の失敗が発生する。「`quote`は文字列を返す」ではS式データを扱えない。

口頭確認

`make-adder` の環境の寿命を聞く。続いて `add7` を別に作り、2つの関数が保存する `x` の違いを追問する。`fact 3` の環境を描かせ、本文を定義時に評価すると何が起きるか聞く。

到達基準案

`define` と `lambda` の役割、特殊形式の評価順序を説明できる。クロージャと名前の隠蔽の反例を通し、別環境を描ける。再帰は停止条件と自己参照の探索先を述べられる。可変長引数など追加機能は本人の仕様に従って確認する。

支援の順序

まず単純な変数、次に引数1個の関数、次に自由変数を持つ返却関数へ進む。定義時と呼出し時のログへ別の印を付けさせる。AIの説明が動的スコープになっていないか、本人が反例を作る。

参照と検証

完成参照は `examples/10-lisp-functions/lisp.py`。 `starter.py` は既存 `square` 組込みへ負数の入力検証を追加するウォームアップ用で、本課題全体の穴埋めではない。

`exercises/10.md` の受入例と、本人の作業用コードの結果を比較する。完成参照を使った範囲を明記させる。

第11回 教員用ノート

授業の位置付け

スクリプト実行と評価器の準備。2026年度の授業計画に沿う新作教材。PythonホストとLisp方言は教材の設計選択。以下の到達基準は教材案であり、正式な採点配分、実際の成績、提出先や締切を規定しない。

進行の重点

式の読取を終端まで反復する入口を作り、同じ環境で評価する。字句解析では文字列内・外の状態を分ける。後半はリスト・型判定・入出力が評価器開発で何に使えるか調べる。

解答と説明

lesson11.lisp の表示は 25 と 49。定義した square を後の式が使う。スクリプトでは途中の式の値を自動表示しないが、CLIは最後の式の値を表示する。この例は 25、49 に加え、末尾の newline の返り値 () を表示する。

- car は (10 20) の先頭 10。
- cdr は残りのリスト (20)。
- cons 5 (10 20) は (5 10 20)。
- total (2 3 4) は 9、空リストは 0。
- "a;b" の ; はコメント開始ではない。
- 存在しないファイルは読取前、括弧不足は読取時、未定義名は評価時の失敗。

参照スクリプトは失敗したら停止し、非ゼロ終了する。REPLのエラー回復と同じ動作だと決めつけない。本人の方針が異なる場合は文書化と結果が一致するか確認する。

よくある誤解

「ファイルは全体を1リストとして読めばよい」では独立した定義の列を誤解する。「リストはすべて関数呼出し」では引用データを扱えない。「記号も文字列もPythonのstrなので同じ」で名前検索とリテラルを区別できない。

口頭確認

`(list (+ 1 2))` と `(quote ((+ 1 2)))` の違いを聞く。結果は `(3)` と `((+ 1 2))`。ファイルで毎回環境を初期化する案の反例を作らせる。読取とOS入出力を評価器から分ける理由を聞く。

到達基準案

複数式のスクリプトを実行し、定義を共有できる。文字列と記号を区別し、リスト再帰の停止条件を説明できる。評価器に必要な操作を対象式と結び付けて述べられる。ホスト依存は現時点の境界として扱う。

支援の順序

2個の数値式、定義と呼出し、明示表示の順に確認する。文字列字句の不具合は空白・;・括弧を1種類ずつ試す。AIのトークン分割案は文字列の反例で確認する。

参照と検証

完成参照は `examples/11-lisp-scripts/lisp.py`。 `starter.py` は既存square組込みへ負数の入力検証を追加するウォームアップ用で、本課題全体の穴埋めではない。

`exercises/11.md` の受入例と、本人の作業用コードの結果を比較する。完成参照を使った範囲を明記させる。

第12回 教員用ノート

授業の位置付け

Lisp製評価器の最小実装。2026年度の授業計画に沿う新作教材。PythonホストとLisp方言は教材の設計選択。以下の到達基準は教材案であり、正式な採点配分、実際の成績、提出先や締切を規定しない。

進行の重点

必要な機能を対象式から導き、Lisp製評価器を最小段階から作る。完成ホストを使って対象評価に集中する経路も認めるが、本人の実装範囲を明記させる。第12回のmeta参照は最小版。対象の完全な名前・関数処理は第13～14回へ接続する。

解答と説明

対象式 7 は 7、`(+ 1 (* 2 3))` は 7。`(quote (+ 1 2))` は `(+ 1 2)`。`(if #f (/ 1 0) 7)` は 7 で非選択枝を評価しない。

ホストに `(quote (quote (+ 1 2)))` を渡すと、ホストが外側の引用を処理して対象式 `(quote (+ 1 2))` を得る。それをLisp製評価器が処理し、対象の結果 `(+ 1 2)` を返す。

環境の表現は連想リストでも別の形式でもよい。検索、定義、拡張の契約をそろえる。対象の名前をホスト環境から検索する実装は、環境の混同になる。組込みの算術をホストへ委譲しても、対象式の分類と引数の評価規則をLispで実装したことを説明する。

よくある誤解

「evalという名前を付ければ評価器」では仕事に分らない。「ホストにlambdaがあるので対象にも完成」では層を混同する。「Lispで書けばPython依存が消える」では実際の起動と読取・入出力を無視する。独立ブートストラップやネイティブ実行は成果に含めない。

口頭確認

数値と未定義記号を同じように返す実装の反例を聞く。次に、二重の引用がどの層で外れるか説明させる。通常の適用でevalとapplyが互いを呼ぶ役割を、対象の関数例で追問する。

到達基準案

数値・引用・四則・分岐の最小評価器が動き、対象とホストを区別できる。不足機能を契約として記述し、次の拡張順序を説明できる。関数の完成を一律に要求せず、第14回までの開発へつなぐ。

支援の順序

まず数値だけ、次に引用、次に二項加算を通す。名前検索やクロージャを一度に加えさせない。対象環境を図に描き、AIの案がホストに丸投げしていないか本人に検証させる。

参照と検証

完成参照は `examples/12-meta-evaluator/lisp.py`。`starter.py` は既存square組込みへ負数の入力検証を追加するウォームアップ用で、本課題全体の穴埋めではない。
`exercises/12.md` の受入例と、本人の作業用コードの結果を比較する。完成参照を使った範囲を明記させる。

第13回 教員用ノート

授業の位置付け

理解確認とディスカッション。2026年度の授業計画に沿う新作教材。PythonホストとLisp方言は教材の設計選択。以下の到達基準は教材案であり、正式な採点配分、実際の成績、提出先や締切を規定しない。

進行の重点

理解確認をディスカッションで行う。講師の連続説明を主活動にせず、予測、ペア説明、実行、反証、説明の更新へ時間を配分する。時間の具体的な分数は授業運用に合わせて決め、正本シラバスの指定として記載しない。

議題の解答

- `(quote missing)` は記号 `missing`。名前を検索しない。
- `(if #f (/ 1 0) 8)` は 8。
- `(if (= 1 1) 9 missing)` は 9。
- `plus2 7` は 9。返却された関数が `x = 2` の環境を保持する。
- 外側の `x = 100` と引数 `x = 4` の例は 5、100。共有辞書へ仮引数を書き込む設計では外側が壊れる。
- 追跡の正常結果は 7。`[host depth=N]` と `[meta depth=N]` を区別する。`meta`もPython製ホスト上で実行するのでホスト側のログが現れる。

予測が誤っていても、実行後に原因を具体的に説明し修正した記録を理解確認の材料にする。最終値だけを当てた参加者には環境や非評価部分を追問する。

よくある誤解

「ログが多ければ説明できる」では環境探索先が不明な場合がある。「動作が同じなら設計も同じ」では追加機能や反例で差が出る。「AIが説明したので理解済み」では本人が初見入力を追えない。

口頭確認と追質問

`make-adder` の `x` がなぜ残るか聞き、異なる2つの加算器を作る入力を追問する。`if` を通常の引数評価で実装する案を出し、反例を求める。Lisp製評価器からPythonホストへの依存を1個挙げさせ、除く場合の追加作業を聞く。

到達基準案

本人が実行前の予測と理由を書ける。他者へ問いを出し、反例またはログで検証できる。説明が変わった場合、その理由を記録できる。未解決の問いに次の検証入力がある。議論の音量や速さを理解の代用にしない。

支援の順序

発言しにくい参加者には、最初に個人メモを書く機会を設ける。ペアの説明が一致したら、講師が新しい小さな入力を1個加える。コードが未完成でも、完成参照と本人の版の差を使って議論できる。

参照と検証

完成参照は `examples/13-language-review/lisp.py`。 `starter.py` は既存square組込みへ負数の入力検証を追加するウォームアップ用で、本課題全体の穴埋めではない。

`exercises/13.md` の受入例と、本人の作業用コードの結果を比較する。完成参照を使った範囲を明記させる。

第14回 教員用ノート

授業の位置付け

Lisp製評価器とREPL。2026年度の授業計画に沿う新作教材。PythonホストとLisp方言は教材の設計選択。以下の到達基準は教材案であり、正式な採点配分、実際の成績、提出先や締切を規定しない。

進行の重点

Lisp製の対象評価器とREPLを実際に動かす。ホストのREPLと `--selfhost-repl` の入口を区別する。第13回で得た反例を、クロージャ・再帰・回復の検証へ使う。

解答と説明

- 対象の `square 6` は 36。
- `make-adder 5` から作った `add5 3` は 8。
- 対象の `fact 4` は 24。
- `(define x 5)` の後、`(+ x 2)`、`(/ 1 0)`、`(+ x 3)` は 7、診断、8。
- EOFと `:quit` は終了。正常な終了は評価失敗と分ける。

対象環境は入力をまたいで保持する。式とエラーは1入力の処理に属する。対象関数の呼出しは、保存した定義時環境を外側にした新しい環境で本文を評価する。

`--meta --eval` は `evaluator.lisp` の `m-eval` を実行する。`--selfhost-repl` は `repl.lisp` の循環を実行する。読取はPython側の`tokenize/read_all`、入出力はホストの組込みを使う。エラー保護 `guard` と反復 `while` もホストへ依存する。本人の実装がどの関数を呼んだか、関連コードと追跡で示す。

よくある誤解

「同じ結果が出たのでLisp製評価器が動いた」では入口の証拠がない。「対象環境は毎回初期化してよい」では定義が次の入力で使えない。「Pythonが起動に関わるだけ」では読取・入出力などの依存を過小に述べる。独立ブートストラップ・ネイティブ実行は達成したと表現しない。

口頭確認

対象とホストに同じ名前がある場合の検索先を聞く。クロージャの環境を描かせ、再帰関数が自分を見つける場所を追問する。エラー後も残す状態と捨てる状態を聞く。読取器をLispへ移すなら必要な機能を検討させる。

到達基準案

Lisp製評価器を呼ぶ実行証拠があり、対象の関数・クロージャ・再帰を説明できる。Lisp製REPLが定義を保持し、エラーから回復し、EOFで終了する。未達部分がある場合は最小の再現入力と開発順序を示せる。達成範囲とホストへの委譲を明記する。

支援の順序

最初は `--meta --eval` で評価器だけを確認する。名前、単純関数、クロージャ、再帰を通してからREPLへ統合する。参照版は1行1式なので、複数行の例は1行にして入力する。

参照と検証

完成参照は `examples/14-selfhosting/lisp.py`。 `starter.py` は既存square組込みへ負数の入力検証を追加するウォームアップ用で、本課題全体の穴埋めではない。 `exercises/14.md` の受入例と、本人の作業用コードの結果を比較する。完成参照を使った範囲を明記させる。

第15回 教員用ノート

授業の位置付け

講評と総括。2026年度の授業計画に沿う新作教材。PythonホストとLisp方言は教材の設計選択。以下の到達基準は教材案であり、正式な採点配分、実際の成績、提出先や締切を規定しない。

進行の重点

開発した実装の講評と総括を主活動にする。新機能の説明を追加する回にはしない。作者の説明、聞き手の反例、結果の観察、改善案の選択をつなぐ。正式な点数配分は本教材で作らない。

架空例の講評

例Aは算術のみだが、対応範囲と失敗条件を明示する架空の案。仕様と実行が一致するか、読取と評価を説明できるかを見る。

例Bは単一の環境へ仮引数を書き込む架空の案。外側 $x = 100$ 、 f の引数 $x = 4$ の反例で外側の破壊を検出する。教材の正しい仕様では $f\ 4$ は 5、呼出し後の x は 100。

例CはPythonホスト上のLisp製評価器。`if` とクロージャの評価をLispで行い、読取と入出力を委譲する架空の案。入口と関連コードを示し、達成範囲を正確に述べたか確認する。

例題ディレクトリの完成実装とfixtureは自作の教材。講義中の架空の不具合設計案を、実際の参照コードにある欠陥として紹介しない。実在の提出物・学生名・成績は使わない。

よくある誤解

「機能数が多いほどよい」では検証や仕様の一致を見落とす。「READMEは後で書けばよい」では再現条件が不足する。「未完成だから説明できない」では部分達成や制限を伝えられない。「AI利用を隠す方がよい」では学習と判断の過程を確認できない。

口頭確認

成功例の結果を予測させ、関連コードと環境を示させる。次に聞き手が反例を1つ加える。改善案には、利用者に起こる変化と確認する入力を追問する。前半の探索・木・計算量がどの処理に使われたか本人のコードで説明させる。

到達基準案

対応機能と制限を証拠で示せる。講評を事実と提案に分けて記録できる。改善案の成功条件を具体的な入力で説明できる。AIに任せた部分と本人が判断・検証した部分を振り返れる。全体総括には本人の判断が変わった具体例がある。

フィードバック案

作者が次の作業へ移れるよう、再現入力と改善箇所を短く伝える。個人の優劣や実在の成績比較を講評材料にしない。授業で決める正式な提出運用と、ここに示した成果物案を区別する。

参照と検証

完成参照は `examples/15-showcase/lisp.py`。`starter.py` は既存square組込みへ負数の入力検証を追加するウォームアップ用で、本課題全体の穴埋めではない。`exercises/15.md` の受入例と、本人の作業用コードの結果を比較する。完成参照を使った範囲を明記させる。